



Zero Trust Application Delivery: How Load Balancers Enforce Identity and Access

Executive Summary

Zero Trust programs at most organizations concentrate on endpoints, identity and the network edge. That work matters, but it leaves a gap at the place where users and services actually meet applications: the route a request travels from a user's device to the application server.

This paper argues that the application delivery layer, specifically the load balancer or Application Delivery Controller (ADC), is a natural Policy Enforcement Point for Zero Trust, and that most organizations already have the infrastructure they need to strengthen their Zero Trust program.

Load balancers sit in the network traffic flow between devices and applications. The load balancers can terminate TLS, understand HTTP/HTTPS and integrate with the same identity providers as the rest of the enterprise IT stack. This positioning lets load balancers take on five essential and related roles:

- Identity awareness through pre-authentication and SSO
- Access control scoped to individual paths and methods
- Traffic inspection via a Web Application Firewall (WAF)
- Continuous verification that responds to changing risk across a session
- Policy enforcement that treats access rules as versioned, declarative code

Together, these roles map cleanly onto the Policy Enforcement Point defined in NIST SP 800-207, a reference architecture that many auditors and security teams already use.

This whitepaper explains how load balancers can enhance Zero Trust security for both human-to-application and service-to-service traffic. Workload identity, mTLS and per-method authorization extend the same model into east-west paths where microservices, internal APIs and shared platforms such as object storage handle most traffic.

We also acknowledge the limits of this layer: the load balancer cannot see into traffic it does not terminate, cannot fix application logic flaws that appear after authorization and introduces a bottleneck that demands careful capacity and availability planning. No single layer closes every gap, and a serious Zero Trust program does not ask any one solution to deliver every aspect of the program.

We highlight example scenarios to show the load balancer Zero Trust pattern in practice: enforcing zone-aware policy on shared object storage so a compromise in a low-trust application cannot spread to high-trust data, and acting as an API gateway for a microservices platform so each service can only call the specific methods its workload identity allows.

In the examples outlined, the applications behind the load balancer do not need modification. Policy resides at the enforcement point where teams can review, do version control and change it without touching either side.

For organizations starting out, the practical path looks like this: identify the capabilities already available on existing load balancers and their licenses, pick a small number of well-understood applications, define policies declaratively, integrate with the existing identity providers, turn on WAF inspection in monitoring mode before enforcing, plan the service-to-service story early and instrument everything from day one to get the telemetry data for analysis.

The deployment pattern matters more than any specific product. An organization that treats its application delivery layer as a Policy Enforcement Point can build the foundation it needs. Capabilities will evolve, but a well-designed architecture will hold and adapt as your needs change.

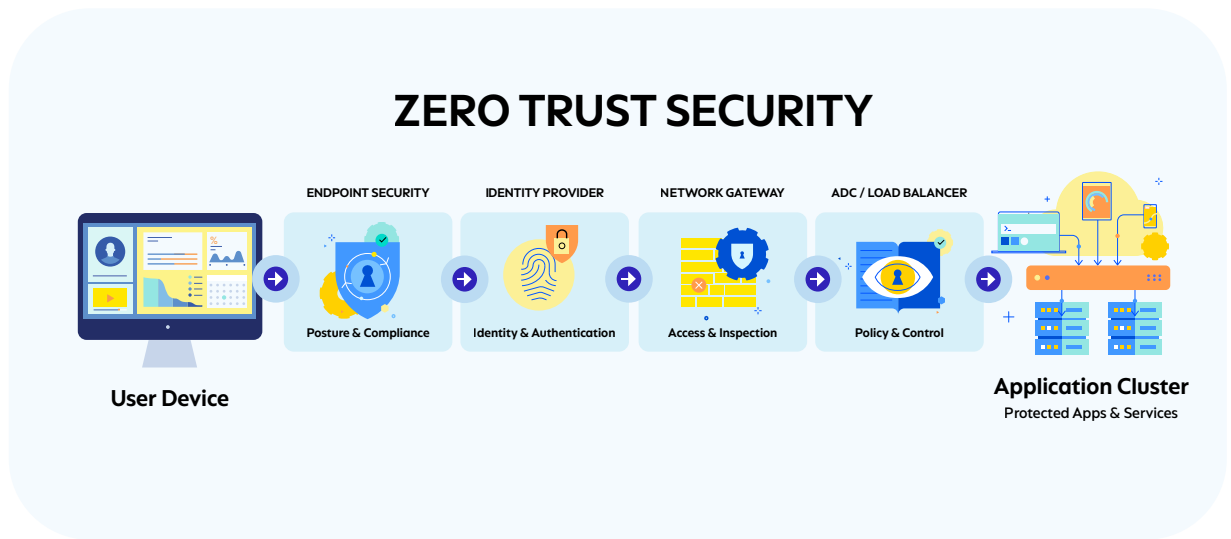
Why the Traffic Path Is Important in Zero Trust

When designing Zero Trust programs, most IT teams start with the endpoint or the network edge. First steps often involve selecting a modern identity/authentication solution and a way to deploy a Zero Trust Network Access (ZTNA) client to devices. Then they dissolve the network perimeter and are now “running Zero Trust.” While those decisions and steps matter, but they don’t address the network traffic paths that client requests follow to communicate with applications and other backend servers.

After a user passes identity and device checks, what prevents them from accessing unauthorized parts of an application? What examines the request before it reaches the backend server? How is a separate policy enforced for a finance API compared with a public marketing page hosted on the same platform? Typically, the answer is “the application,” which often reflects whatever each development team managed to implement.

This paper argues that the application delivery layer, particularly the load balancer, is underused in Zero Trust deployments. Positioned in the request pathway, it already handles TLS termination, understands HTTPS and maintains identity context via integration with identity providers. Incorporating policy enforcement at this layer helps address a gap that endpoint and network controls alone cannot close.

The following sections define Zero Trust Application Delivery and connect it to the NIST reference architecture, which is familiar to auditors and security teams. We explain why traffic flow is important, describe the load balancer's role in both user access and service-to-service communication and discuss the limitations of endpoint-only and VPN-based solutions. The discussion also explains what the load balancer layer cannot achieve, walks through a conceptual reference architecture and concludes with practical advice for organizations to begin implementing these strategies.



What Zero Trust Application Delivery Means

Zero Trust is a comprehensive architectural approach that encompasses identity, devices, networks, data, workloads and the analytics linking them.

Zero Trust Application Delivery focuses specifically on how requests reach an application and their journey afterward. The key point is how these principles translate into actual behavior in the request process. For example, how each request is explicitly verified, regardless of origin. How access decisions are per-request and at the resource level rather than across the entire application and how inspection extends beyond login.

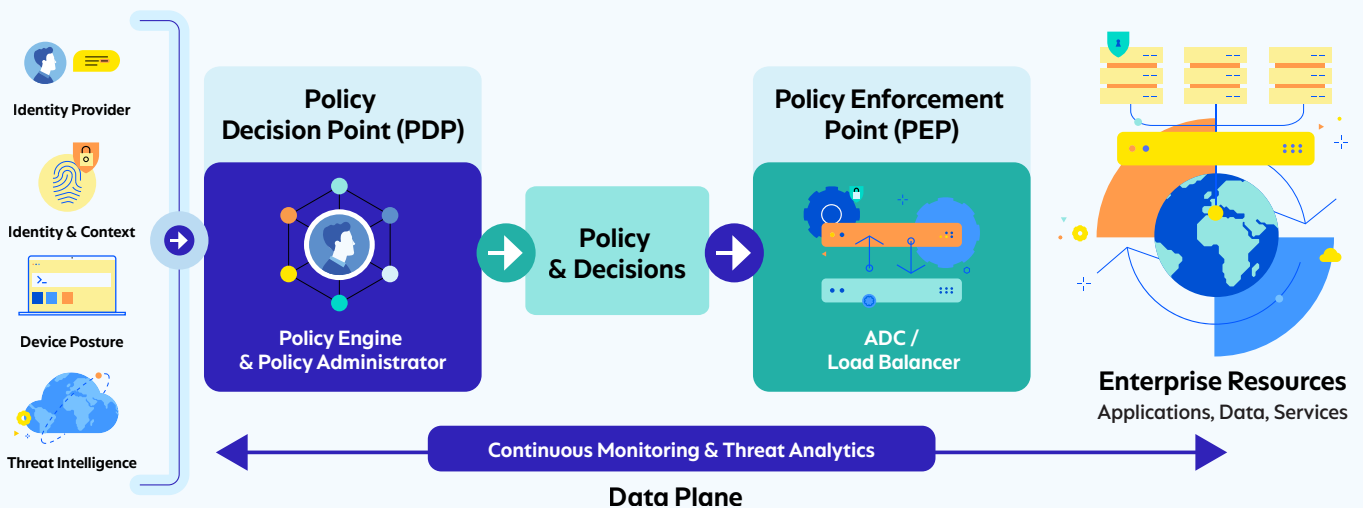
Zero Trust Application Delivery implements these principles directly within the traffic flow, before requests reach the application. It treats each request as requiring authentication, authorization, inspection and routing in accordance with policies. The load balancer functions as the point where all these actions occur.

Mapping to NIST SP 800-207

The NIST Zero Trust Architecture outlines a small set of key logical components fundamental to most modern Zero Trust approaches. A Policy Decision Point (PDP) assesses access requests against policy, while a Policy Enforcement Point (PEP) enforces those decisions within the data flow. A Policy Administrator manages the interaction between the two, using data from sources such as identity providers, device posture services and threat intelligence feeds to inform decisions.

In Zero Trust Application Delivery, the load balancer acts as the Policy Enforcement Point. Positioned in the request path, it terminates sessions and enforces decisions. The Policy Decision Point may reside within the load balancer for simple policies or in a separate policy engine that it consults. Clear role naming helps teams map the architecture to familiar frameworks recognized by auditors and aligns with Zero Trust maturity models increasingly demanded by regulators.

NIST SP 800-207 Zero Trust Architecture



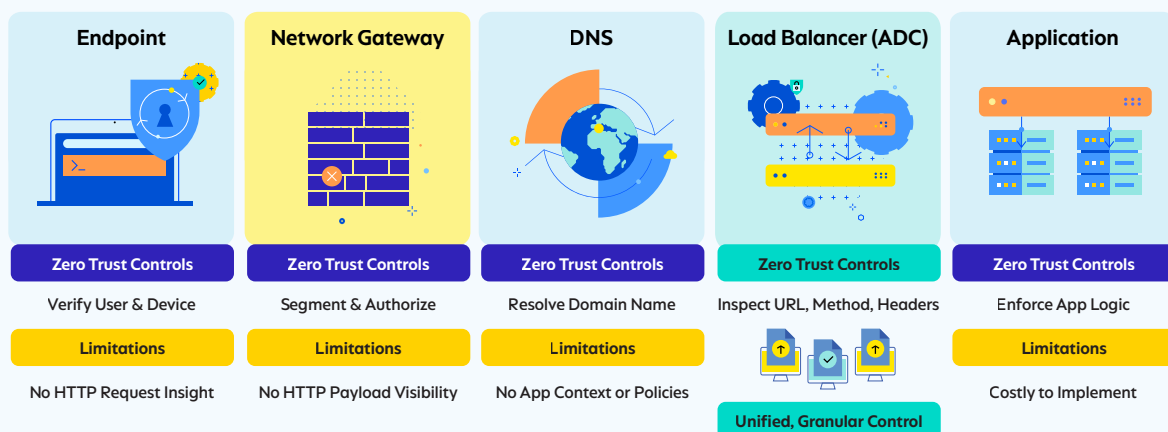
Where Zero Trust Controls Sit in the Traffic Path

In a typical application delivery setup, a user request passes through multiple layers before reaching the backend service. An endpoint agent might verify the device. A network gateway or ZTNA broker could check identity and authorize access to a network segment. DNS resolves the domain name. The request then reaches a load balancer, which forwards it to a backend pool. Finally, the application processes the request. Each of those layers can carry Zero Trust responsibility. The question is which layer best handles each control.

Endpoint and identity layers effectively address “Who is this and what device are they using.” However, they do not clarify “What is this particular HTTP request attempting to do, and whether this user should perform this action on this specific resource now.” Network layers manage segmentation and traffic admission between zones. Typically, they do not inspect HTTP payloads, lack understanding of application semantics and do not base decisions on URL paths or HTTP methods.

The application itself can enforce fine-grained authorization. This approach places the burden on every application team to implement consistent identity, session and authorization logic across all languages and frameworks for every workload. In practice, coverage is uneven, configuration drifts and legacy applications often resist modification altogether.

Where Zero Trust Controls Sit in the Traffic Path



The load balancer occupies a strategic position as the last common checkpoint before the backend servers. It terminates TLS, which allows it to see the contents of HTTPS requests rather than just the packet headers. In turn, the load balancer already knows the URL, method (GET, POST, etc.), headers and body. It also integrates with the same identity providers as the rest of the stack. By enforcing policies uniformly across applications, it eliminates the need to modify each application individually.

Enforcing Zero Trust controls at the application delivery layer does not replace endpoint or network controls. It complements them by closing the gap between “the user got onto the network” and “the user is accessing a specific resource in a specific application.”

The Role of the Load Balancer in Zero Trust

Modern load balancers do far more than distribute traffic across servers. In a Zero Trust Application Delivery model, they take on five related roles.

Identity Awareness

The load balancer integrates with the organization’s identity providers, whether those are on-premises directory services, cloud identity platforms or a mixture of both. It pre-authenticates users before any request reaches the backend servers. Single sign-on, multi-factor authentication, certificate-based authentication and smart card or common access card authentication can all enforce authentication at this layer.

This matters for two reasons. First, it removes the assumption that a user inside a network segment is automatically authenticated. Second, it gives the load balancer something useful to work with when making per-request decisions: a verified identity, group memberships and session attributes that policy can act on.

Access Control

Once the load balancer identifies a user, it determines what they can access. This goes well beyond “can this user access this application?” Policies apply to individual paths, methods and resources within the application. A user in one group may read from an API endpoint, while a user in another group may also write to it. A third group may get denied access entirely. Source network, geography, time of day and other contextual signals all feed into access decisions. The result is application access that follows least privilege at the request level rather than at the application level.

Inspection

Because the load balancer terminates TLS, it inspects the contents of requests rather than passing encrypted bytes untouched. A Web Application Firewall (WAF) at this layer applies rule sets such as the OWASP Top 10 to block common exploits, enforces custom rules for application-specific patterns, draws on IP reputation feeds to block known bad actors and applies geographic restrictions where appropriate. Inspection also supports the “assume hostile” principle. Even a request from an authenticated, authorized user may contain an injection attempt, a malformed payload or behavior that does not match the application’s expectations.

Continuous Verification

Zero Trust does not stop at login. The load balancer supports verification that continues throughout a session. Short-lived tokens require revalidation at regular intervals. Risk signals from the identity provider or endpoint posture service can trigger step-up authentication for sensitive operations. A session that looked fine at sign-in can get challenged when the user tries to delete records, access financial data or change permissions. If device posture changes mid-session, such as when a device falls out of compliance, or the identity provider flags anomalous behavior, the load balancer can drop the session or force re-authentication rather than let the original login carry trust forward indefinitely.

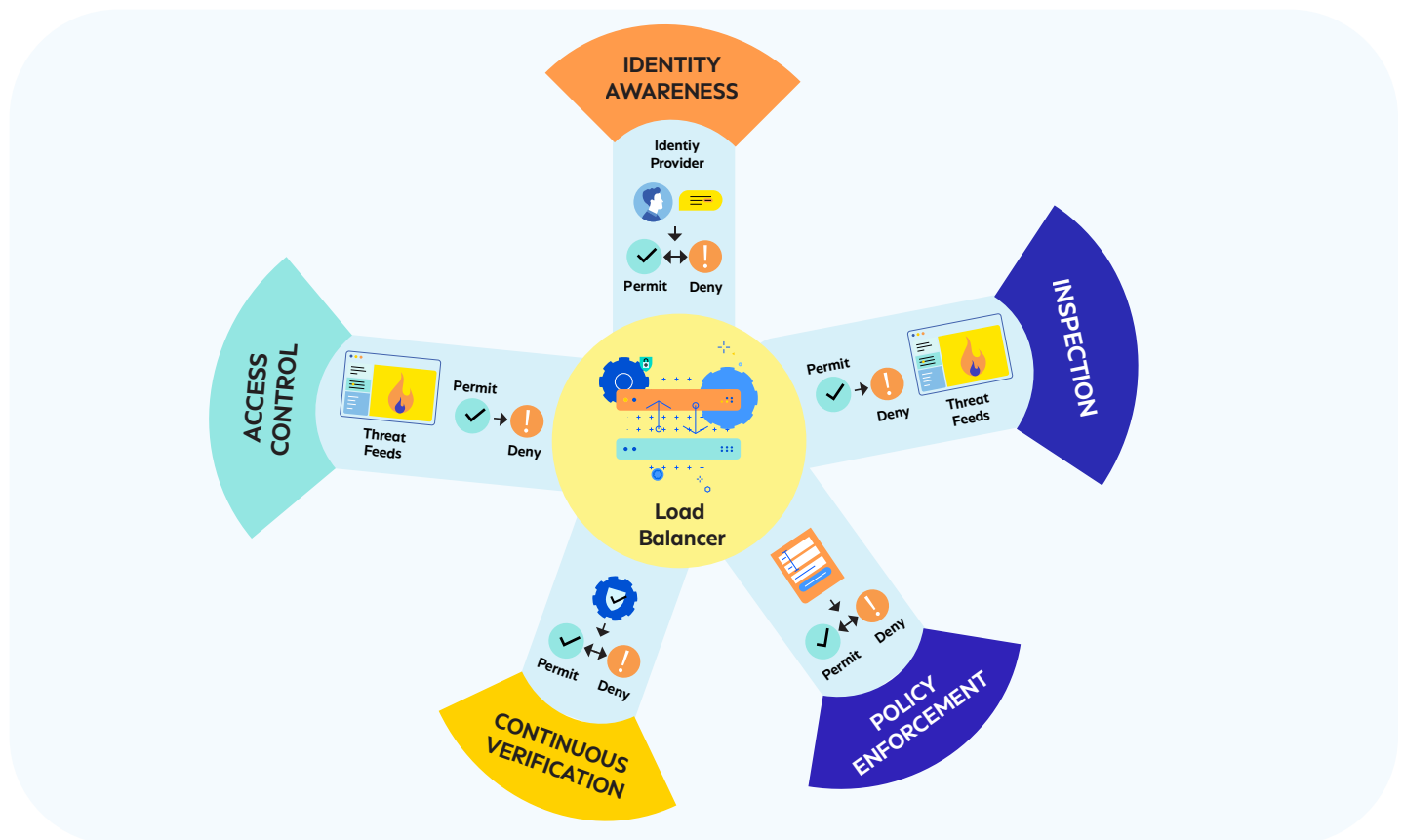
This is where the “continuous” part of continuous verification lives in the traffic path. Endpoint agents observe the device. Identity providers score the user. The load balancer turns those signals into enforcement.

Policy Enforcement

Identity, access control, inspection and continuous verification come together as policy. A well-designed load balancer treats policy as a first-class object: declared, versioned, consistently applied and observable. Teams define policy as code, deploy it automatically and update it without rewriting the underlying application.

This is what makes the load balancer a practical enforcement point. Each application team does not need to reinvent identity and authorization. The platform team defines policy once, applies it across published applications, and adjusts it as the threat landscape and the business changes.

Taken together, these five roles transform the load balancer from a traffic distributor into a Policy Enforcement Point for application access. The applications behind it do not need to change. Their security posture improves because everything that reaches them has already passed authentication, authorization, inspection and policy-based routing.

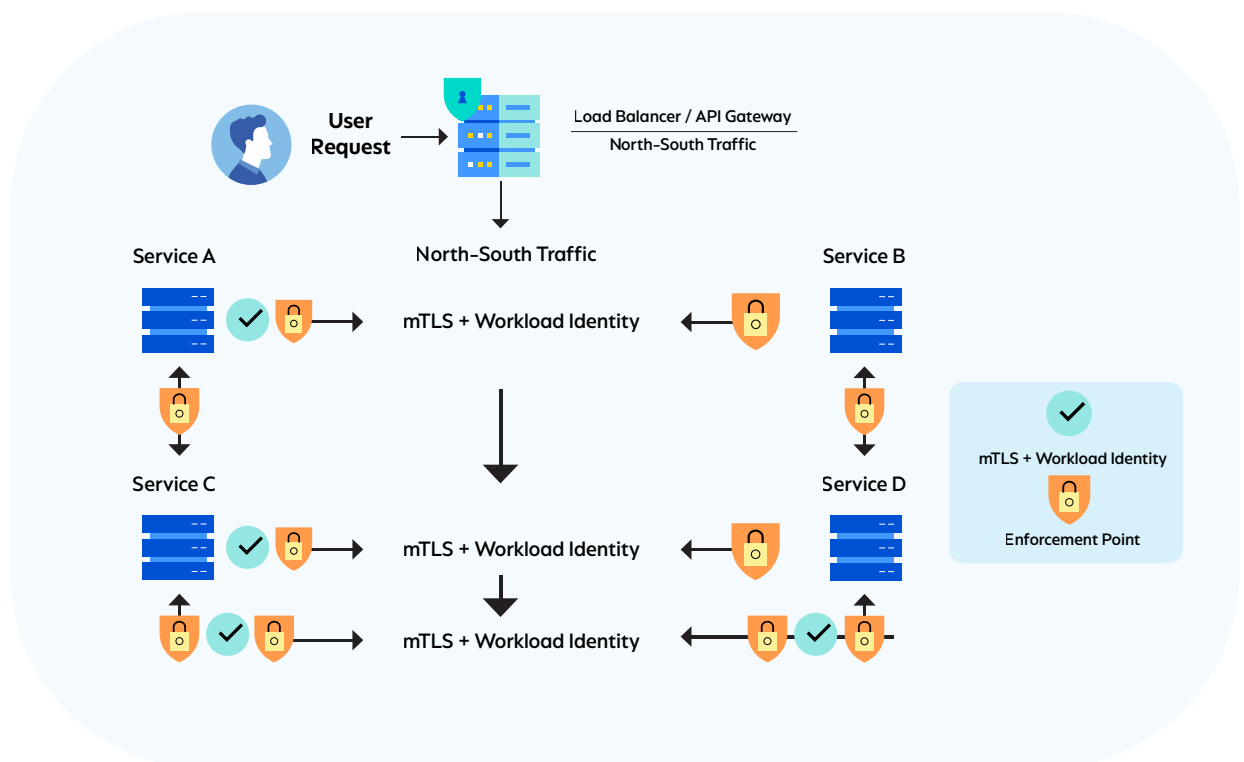


Service-to-Service Traffic and Machine Identity

So far, we've focused on describing requests from human users. Modern applications rarely look like that in end-to-end use. A user request often triggers an API call within an application. That API calls another service over the network. That service then reads from a database, publishes a message to a queue and then calls another third-party API. The bulk of traffic inside a modern application platform is machine-to-machine, not human-to-machine or machine-to-human.

Zero Trust that stops at human interaction leaves this traffic unprotected. Historically, most organizations accepted this because east-west traffic was assumed to be trusted. That assumption is exactly what Zero Trust aims to replace.

The application delivery layer also plays a role here. Workload identity, typically issued as short-lived certificates or SPIFFE identities, enables services to authenticate with each other using cryptographic guarantees rather than relying on shared secrets or network location. Mutual TLS (mTLS) provides both sides of a connection with verified identities. API gateways and load balancers can enforce that only a named service that presents a valid workload identity can call a specific API method on a specific endpoint. The same per-request authorization model that applies to users also applies to services.



Not every load balancer deployment needs to shoulder the full weight of service mesh responsibilities. Some organizations run a service mesh for east-west traffic and use the load balancer for north-south traffic. Others extend the load balancer to selective east-west paths where it adds value, particularly at zone boundaries and in front of shared services such as object storage, databases or identity APIs. What matters is that the architecture treats service identity as a first-class concern rather than an afterthought.

Common Gaps in Endpoint-Only or VPN-Based Zero Trust

Many organizations have made real progress on Zero Trust at the endpoint and network layers. That work is valuable, but it is incomplete on its own. A few patterns that impact the end-to-end trust relationships recur. They are:

The authenticated tunnel gap. A ZTNA client or a VPN authenticates the user and grants access to an application or a network segment. From that point on, the connection carries trust. Whatever the user does using that connection becomes the application's problem. If the application has weak internal authorization or an attacker hijacks the user's session, nothing in the connection itself will detect it.

The all-or-nothing gap. Network-level access controls grant or deny access to an entire application or service. They do not naturally express rules such as "this group can read from this data but not write to it" or "this service can call these three API endpoints but not the others." Enforcing that level of granularity at the network layer is either not possible or fragments the network into so many segments that it becomes unmanageable.

The encrypted blind spot. Network controls that do not terminate TLS cannot see what an inside requests is. They route traffic and block by source or destination, but they cannot distinguish a normal request from a SQL injection attempt or a malformed payload designed to exploit a known vulnerability.

The every-application-for-itself gap. Pushing all authorization logic into individual applications leaves the organization's security posture at the mercy of whatever each development team has time to implement. Coverage varies. Configurations drift. Legacy applications often resist modification altogether. Acquired applications arrive with whatever the previous owner deemed good enough.

The east-west blind spot. Traffic between services, particularly in microservices and containerized environments, often flows unprotected, based on the assumption that anything already inside is trusted. This shouldn't happen in a Zero Trust organization.

None of these recurring patterns implies that the endpoint and network VPN-based Zero Trust controls are faulty. They are essential, but alone do not suffice. Bridging these gaps requires a layer that understands the application protocol, responds to identity and request context for each request and enforces uniform policies across applications without relying on each application development team to implement them.

What the Load Balancer Cannot Do

An honest Zero Trust architecture recognizes the limits of each enforcement point, including the load balancing layer. Three things the load balancer cannot do on its own are worth noting.

It cannot inspect traffic that it does not terminate. If an application uses end-to-end encryption that bypasses the load balancer, or if it forwards traffic without terminating TLS, inspection becomes ineffective. Some applications require end-to-end encryption for legitimate reasons, necessitating alternative enforcement mechanisms, typically within the application itself or via a sidecar proxy that terminates TLS on the application's behalf.

It can't magically fix application logic issues. A load balancer can verify a user's permission to access an endpoint, but cannot fix application logic flaws that happen after authorization. Problems such as broken object-level authorization and business-logic errors are beyond the scope of traffic-layer controls. As a result, secure development practices, detailed code reviews, and focused application security testing remain crucial.

It introduces a bottleneck. A Policy Enforcement Point that sees every request is also a failure risk and a capacity consideration bottleneck. Modern load balancers are built for this role with high availability, horizontal scaling and performance optimizations, but the architectural decision is real. TLS termination, WAF inspection and per-request policy evaluation all cost cycles. Organizations deploying via this model should plan for redundancy, understand the latency envelope of their policy and instrument the path carefully.

Recognizing these limits does not argue against using load balancers within the Zero Trust architecture. It is an argument for clear thinking about what each layer and component contributes. The load balancer closes real gaps. It does not close every gap and no serious Zero Trust program should depend on any single solution.

A Reference Architecture

In this section, we describe a conceptual reference architecture for Zero Trust Application Delivery. This is not a deployment guide, but rather we aim to show all the pieces needed to deliver secure applications within a broader Zero Trust program.

The identity provider issues and validates user identities. It manages group memberships, supports multi-factor authentication and serves as the source of truth for user identity. This is typically a cloud identity platform, an on-premises directory service or a federation of both.

The device posture and endpoint service provides real-time signals about the health, compliance and risk score of the device originating each request. The load balancer consults these signals when making access decisions and when deciding whether to trigger authentication challenges.

The load balancer sits in the request path between users and applications, serving as the Policy Enforcement Point. It pre-authenticates users against the identity provider. It enforces access control policies based on identity, group membership, source context, device posture and request attributes. It inspects requests using a Web Application Firewall. It steers traffic to the appropriate backend pool based on policy. For service-to-service calls across zone boundaries, it enforces workload identity via mTLS.

The policy engine, whether embedded in the load balancer or running alongside it, serves as the Policy Decision Point. Simple rules can reside in the load balancer itself. More complex policies, particularly those that draw on external context such as threat intelligence or risk scoring, benefit from a dedicated

engine that the load balancer calls.

The published applications sit behind the load balancer. They may be traditional web applications, APIs, microservices running in Kubernetes, object storage endpoints or any combination of these. They do not need to be modified to participate in this model. They receive only the traffic that policies permit.

The telemetry and management layer observes what the load balancer is doing: which rules trigger, which requests get blocked, which authentications succeed or fail, where traffic originates and how policy performs over time. This layer is essential. Zero Trust without observability is a configuration exercise, not a security posture.

Here are two examples to make this more concrete.

Example One: Shared Object Storage Across Security Zones

Consider an organization that provides shared object storage to multiple internal applications. Applications reside in different security zones, defined by the source network. All of them need access to some shared data, but the organization needs hard guarantees that an application in a low-trust zone cannot write to a data store that a high-trust application uses. The threat model is straightforward: a compromised low-trust application uploads a malicious object, and a high-trust application later processes it, spreading the compromise.

In the reference architecture, the load balancer enforces this directly. The policy is declarative and defined once. It maps source zones to permitted operations on specific data stores: read-only from the low-trust zone, read and write from the medium-trust zone and full access, including delete, from the high-trust zone. Every request is evaluated against that policy before it reaches the storage back end. The applications themselves do not need to enforce zone-aware authorization. The storage platform does not need modification. The policy lives at the enforcement point, where teams can review, version control and change it without touching either side.

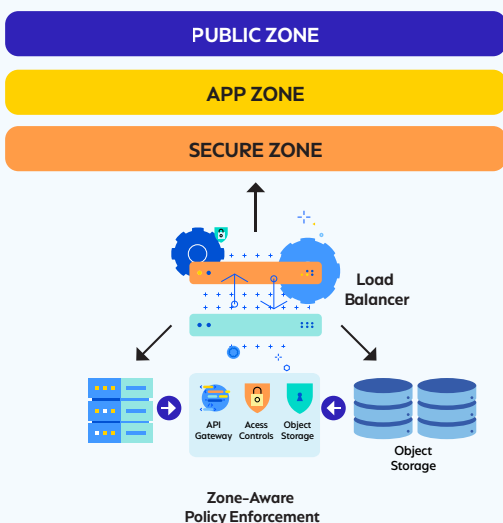
Example Two: Internal API Gateway for a Microservices Platform

A platform team runs dozens of internal microservices on Kubernetes. Some services handle customer data, some handle financial transactions and some are purely operational. Today, any service that can reach the cluster network can call any other service's API. A compromise anywhere spreads everywhere.

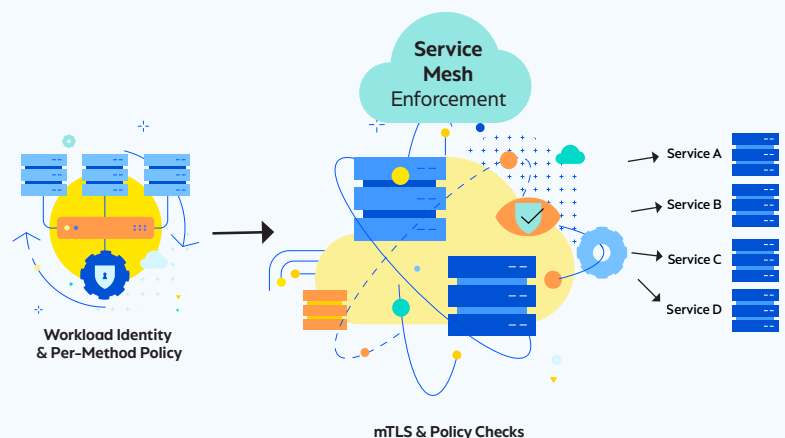
The load balancer sits in front of the service mesh as the Policy Enforcement Point for inter-zone and external API traffic. Each service has a workload identity issued by the platform. Policies specify which services can call which API methods on which other services. The billing service can read from the customer service but cannot write to it. The reporting service can read from both, but cannot call the payments service. External API consumers, authenticated via OAuth, see only the endpoints permitted by their authentication scope.

The applications themselves do not implement this authorization. The mesh and the load balancer do. When deploying a new service, its policy is defined alongside its Kubernetes Helm chart and reviewed as part of the change. When a service is compromised, the threat radius is limited to what that service's identity was allowed to do.

Object Storage Access



Microservices Security



How To Get Started with Zero Trust Application Delivery

A full Zero Trust Application Delivery rollout is a program, not a project. Most organizations do best by starting with a small set of applications, learning what policy looks like in their environment and then expanding from there. A few practical steps follow.

Identify the load balancers already in your environment. Most organizations already run load balancers in front of their important applications. The first question is: what can those platforms already do that is not currently enabled? Identity provider integration, pre-authentication, WAF and access control are often included in existing licenses but go unused.

Pick a small number of applications to start with. Choose applications with a well-understood access model and where the consequences of getting policy wrong are low risk. Internal applications with a clear user base often make a good starting point. So do applications where existing access control is known to be poor and where the business has been asking for something better.

Define policy declaratively. Treat access policy as code. Write it in a form that teams can review, version control and deploy automatically. Avoid policies that exist only in someone's memory or in screenshots of an admin interface. Declarative policy is what makes Zero Trust auditable and repeatable.

Integrate with the identity provider you already have. Do not set up a parallel identity system for application access. The load balancer should consume identity from the same source that the rest of the organization uses. This keeps group memberships, MFA enforcement and user management consistent.

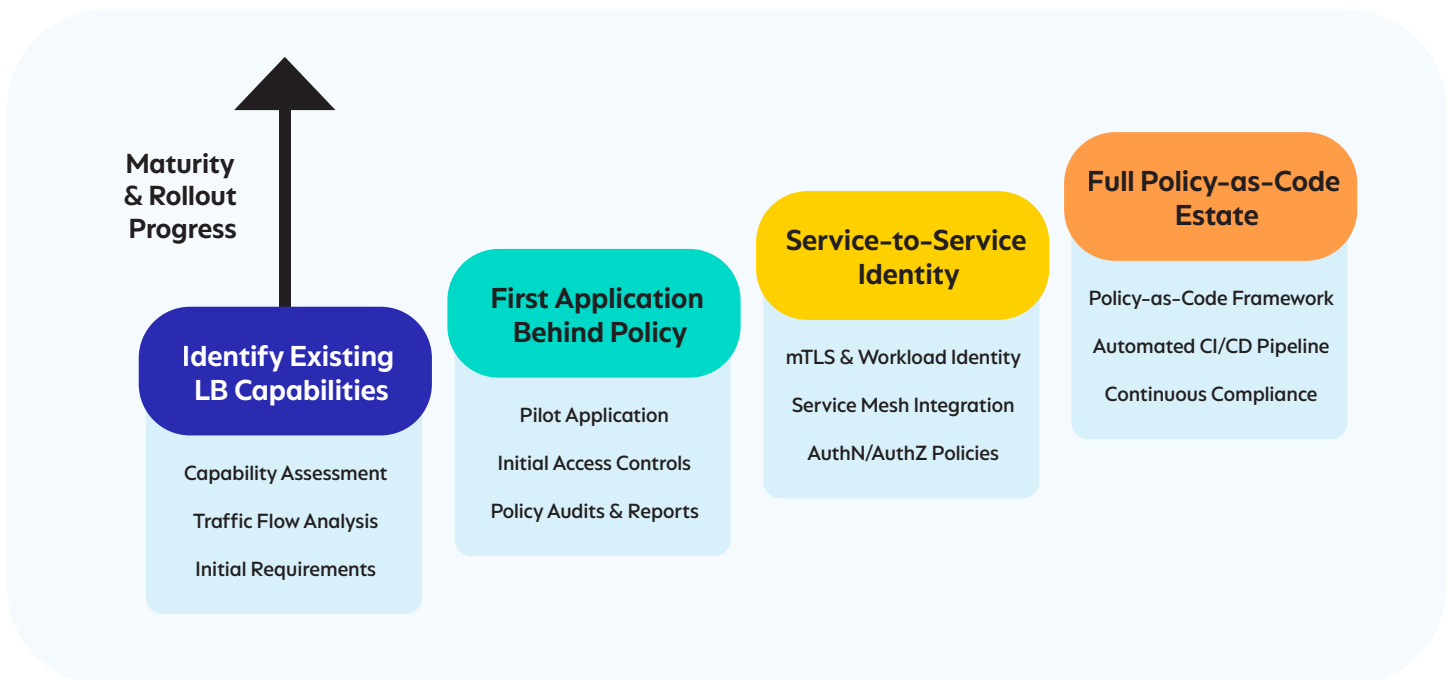
Turn on inspection where it makes sense. A WAF in front of a web application or API helps block a significant share of common attacks with minimal tuning. Start in monitoring mode that only flags issues, observe which triggers get flagged, tune the settings to remove any false positives and then turn on enforcement to block traffic.

Instrument everything for telemetry data collection. Zero Trust depends on knowing what is happening. Put a telemetry layer in place from day one to record which rules fire, which authentications fail, where traffic originates and what gets blocked. Without this, policy decisions become guesses.

Plan the service-to-service path. If your environment includes microservices, containers or significant API traffic, decide early on how the security team will implement workload identity and the location of enforcement points. Retrofitting this later is harder than building it in from the start.

Plan to extend the model. Once the pattern works for one set of applications, the same load balancer can apply the model to others. Microservices in Kubernetes, object storage, internal APIs, partner integrations and traditional web applications can all sit behind the same enforcement point, each with policies appropriate for its use case.

The pattern matters more than any specific product. An organization that treats its application layer load balancers as Policy Enforcement Points can define policies declaratively, integrate with its existing identity provider and record events via telemetry. Specific capabilities and integrations will evolve, but a robust core architecture will hold over time.



A Note on Enforcement in Practice

Multiple load balancer platforms support the patterns described in this white paper. The Progress® Kemp® LoadMaster® solution brings most of these capabilities together in an easy-to-use way via identity provider integration, multi-factor authentication and SSO, a WAF with the OWASP Top 10 core rule set, IP reputation and geographic controls and a Zero Trust Access Gateway that applies declarative policy to published applications. LoadMaster 360 provides the telemetry and management layer that makes the resulting policies observable in use.

The point is not that any one product is all that's needed. Rather, the architecture described above is achievable today on platforms that organizations already use. Zero Trust Application Delivery does not require a new appliance category. It requires using the application delivery layer that already sits in the request path as the Policy Enforcement Point on your load balancers.

Final Thoughts

Zero Trust at the endpoint and the network is necessary work. Zero Trust at the application delivery layer closes the gap between “the user is authenticated” and “the user is doing something they are allowed to do, against a request that is safe to process.”

The load balancer has long occupied a prime position on the network. Treating it as a Policy Enforcement Point, rather than just a traffic distributor, is a high impact move by security and platform teams.

The architecture for delivering Zero Trust Application Delivery within a broader Zero Trust program is not new. The capabilities are not outlandish or hard to implement. What is new is the recognition that the application delivery layer is where Zero Trust principles meet the actual applications, request by request, service by service, day after day.

References

- NIST Special Publication 800-207: Zero Trust Architecture
- OWASP Top 10 Web Application Security Risks
- SPIFFE (Secure Production Identity Framework for Everyone) specifications
– <https://spiffe.io>
- Progress Kemp LoadMaster documentation
- LoadMaster 360 telemetry and management documentation
- Progress LoadMaster Zero Trust Webinar
- LoadMaster Zero Trust Access Gateway



Learn how load balancers act as Zero Trust policy enforcement points to verify identity, enforce access control, inspect traffic, and secure application delivery

About Progress Software

Progress Software (Nasdaq: PRGS) empowers organizations to achieve transformational success in the face of disruptive change. Our software enables our customers to develop, deploy and manage responsible AI-powered applications and personalized digital experiences with agility and ease. Businesses of all sizes get a trusted provider in Progress, with the products, expertise and vision they need to turn AI disruption into a competitive advantage. Millions of developers and technologists at hundreds of thousands of organizations depend on Progress every day. Learn more at www.progress.com

© 2026 Progress Software Corporation and/or its subsidiaries or affiliates.
All rights reserved. Rev 2026/06 | RITM0368413

Worldwide Headquarters

Progress Software Corporation
15 Wayside Rd, Suite 400, Burlington, MA 01803, USA
Tel: +1-800-477-6473

 facebook.com/progresssw
 x.com/progresssw
 youtube.com/progresssw
 linkedin.com/company/progress-software
 [progress_sw_](https://instagram.com/progress_sw_)